

AI Resume Analyzer & RAG Skill Agent

Developer: Shayan Ali
Mentor:Anzal Hussain

July 25, 2026

Contents

1	Executive Summary	2
2	System Architecture	2
3	Modular Directory Structure	2
4	Detailed Module Implementations	2
4.1	Configuration (<code>config.py</code>)	2
4.2	Data Validation Schemas (<code>schemas.py</code>)	3
4.3	Document Ingestion Engine (<code>document_loader.py</code>)	3
4.4	LLM Analysis & RAG Pipeline (<code>llm_engine.py</code>)	4
4.5	Vector Store Setup (<code>rag_store.py</code>)	5
5	Execution and Deployment Guide	6
5.1	Environment Setup	6
6	Acknowledgements	6

1 Executive Summary

The **AI Resume Analyzer & RAG Skill Agent** is an end-to-end intelligent Applicant Tracking System (ATS) optimization tool. It evaluates candidate resumes against target job descriptions using Large Language Models (LLMs) and provides structured feedback regarding match percentage, identified skills, and missing technical competencies. Furthermore, it integrates a Retrieval-Augmented Generation (RAG) pipeline backed by a vector database to automatically query and suggest targeted learning roadmaps for missing skills.

2 System Architecture

The system follows a modular micro-architecture built entirely on top of the LangChain ecosystem, Streamlit, and ChromaDB.

- **Frontend Presentation Layer:** Streamlit web interface for document uploads, job description input, and metric rendering.
- **Ingestion Engine:** LangChain PyPDFLoader for structured PDF document parsing.
- **LLM Execution Engine:** LangChain Expression Language (LCEL) chain executing Groq's LLaMA 3.3 model.
- **Validation Layer:** Pydantic schemas enforcing strict JSON structures and key aliases.
- **RAG Storage Layer:** In-memory ChromaDB vector store leveraging HuggingFace embeddings (all-MiniLM-L6-v2) for semantic course retrieval.

3 Modular Directory Structure

The repository is modularized into dedicated components to maintain clean separation of concerns:

```
Resume_Analyzer/  
  config.py           # Project constants and configuration parameters  
  schemas.py          # Pydantic structured output models  
  document_loader.py  # LangChain PDF parsing routines  
  llm_engine.py       # LCEL analysis and RAG chains  
  rag_store.py        # Vector store initialization and embeddings  
  app.py              # Main Streamlit dashboard application  
  requirements.txt     # Dependency specifications
```

4 Detailed Module Implementations

4.1 Configuration (config.py)

Defines globally accessible settings such as embedding model choices and LLM identifiers.

```

1 EMBEDDING_MODEL_NAME = "all-MiniLM-L6-v2"
2 GROQ_MODEL_NAME = "llama-3.3-70b-versatile"
3 VECTOR_COLLECTION_NAME = "learning_resources"

```

Listing 1: config.py

4.2 Data Validation Schemas (schemas.py)

Implements the Pydantic data model enforcing structured output generation. It utilizes AliasChoices to gracefully handle minor variations in LLM key outputs.

```

1 from pydantic import BaseModel, Field, AliasChoices
2 from typing import List
3
4 class ResumeAnalysisSchema(BaseModel):
5     match_score: int = Field(
6         validation_alias=AliasChoices("match_score", "match_percentage"
7     ),
8         description="Match percentage between 0 and 100 based on job requirements"
9     )
10    found_skills: List[str] = Field(
11        validation_alias=AliasChoices("found_skills", "matching_skills"
12    ),
13        description="Skills present in both candidate CV and Job Description"
14    )
15    missing_skills: List[str] = Field(
16        description="Critical skills present in Job Description but missing from CV"
17    )
18    recommendations: List[str] = Field(
19        validation_alias=AliasChoices("recommendations", "relevant_experience", "suggestions"),
20        default_factory=list,
21        description="Actionable tips on how to improve the CV"
22    )

```

Listing 2: schemas.py

4.3 Document Ingestion Engine (document_loader.py)

Handles raw PDF uploads by creating temporary file buffers and extracting text using LangChain's PyPDFLoader.

```

1 import os
2 import tempfile
3 from langchain_community.document_loaders import PyPDFLoader
4
5 def load_pdf_with_langchain(uploaded_file) -> str:
6     """Uses LangChain PyPDFLoader to read unstructured PDF resumes."""
7     with tempfile.NamedTemporaryFile(delete=False, suffix=".pdf") as tmp_file:
8         tmp_file.write(uploaded_file.getvalue())
9         tmp_path = tmp_file.name
10
11     loader = PyPDFLoader(tmp_path)

```

```

12 docs = loader.load()
13 os.remove(tmp_path)
14 return "\n".join([doc.page_content for doc in docs])

```

Listing 3: document_loader.py

4.4 LLM Analysis & RAG Pipeline (llm_engine.py)

Contains the core LCEL execution logic. The prompt templates utilize escaped double curly braces `{{...}}` to prevent prompt variable interpolation issues during JSON schema definition.

```

1 from typing import List
2 from langchain_core.prompts import ChatPromptTemplate
3 from langchain_core.output_parsers import StrOutputParser
4 from langchain_groq import ChatGroq
5 from schemas import ResumeAnalysisSchema
6 from config import GROQ_MODEL_NAME
7
8 def run_analysis_chain(cv_text: str, job_desc: str, api_key: str) ->
ResumeAnalysisSchema:
9     llm = ChatGroq(model=GROQ_MODEL_NAME, temperature=0, groq_api_key=
api_key)
10     structured_llm = llm.with_structured_output(ResumeAnalysisSchema,
method="json_mode")
11
12     prompt = ChatPromptTemplate.from_messages([
13         ("system",
14          "You are an expert ATS software and Technical Recruiter.\n"
15          "Compare candidate CVs against Job Descriptions.\n"
16          "You MUST output valid JSON containing EXACTLY these keys:\n"
17          "{{\n"
18           '  "match_score": 75,\n'
19           '  "found_skills": ["Python", "Pandas"],\n'
20           '  "missing_skills": ["Docker", "LangGraph"],\n'
21           '  "recommendations": ["Add project examples for Docker"]\n'
22          "}}"),
23     ],
24     ("human", "Candidate CV:\n{cv_text}\n\nJob Description:\n{
job_desc}")
25     ])
26
27     chain = prompt | structured_llm
28     return chain.invoke({"cv_text": cv_text, "job_desc": job_desc})
29
30 def run_rag_recommendations(missing_skills: List[str], vectorstore,
api_key: str):
31     retriever = vectorstore.as_retriever(search_kwargs={"k": 2})
32     llm = ChatGroq(model=GROQ_MODEL_NAME, temperature=0.2, groq_api_key
=api_key)
33
34     recommendations = []
35     for skill in missing_skills[:4]:
36         docs = retriever.invoke(f"How to learn {skill}")
37         context_str = "\n".join([f"- {d.page_content} (Source: {d.
metadata.get('resource')})" for d in docs])
38

```

```

39     rag_prompt = ChatPromptTemplate.from_template(
40         "Target Skill to Learn: {skill}\n"
41         "Retrieved Context:\n{context}\n\n"
42         "Write a short, encouraging 2-sentence study plan."
43     )
44     rag_chain = rag_prompt | llm | StrOutputParser()
45     advice = rag_chain.invoke({"skill": skill, "context":
context_str})
46     recommendations.append({"skill": skill, "advice": advice, "
sources": docs})
47
48     return recommendations

```

Listing 4: *llm_engine.py*

4.5 Vector Store Setup (*rag_store.py*)

Initializes an in-memory Chroma vector store populated with technical course roadmaps and converts them to high-dimensional embeddings using HuggingFace sentence transformers.

```

1  import streamlit as st
2  from langchain_huggingface import HuggingFaceEmbeddings
3  from langchain_chroma import Chroma
4  from langchain_core.documents import Document
5  from config import EMBEDDING_MODEL_NAME, VECTOR_COLLECTION_NAME
6
7  @st.cache_resource
8  def initialize_vector_store():
9      embeddings = HuggingFaceEmbeddings(model_name=EMBEDDING_MODEL_NAME)
10     knowledge_docs = [
11         Document(page_content="LangGraph: Master stateful multi-agent
workflows.", metadata={"topic": "LangGraph", "resource": "LangChain
Academy"}),
12         Document(page_content="Docker & Containers: Learn
containerization and Kubernetes basics.", metadata={"topic": "Docker
", "resource": "Docker Docs"}),
13         Document(page_content="FastAPI: Production-ready Async Python
APIs.", metadata={"topic": "FastAPI", "resource": "FastAPI Official
Tutorial"}),
14         Document(page_content="Vector Databases: Learn ChromaDB,
Pinecone, and similarity search.", metadata={"topic": "ChromaDB", "
resource": "Pinecone Learning Center"}),
15     ]
16
17     return Chroma.from_documents(
18         documents=knowledge_docs,
19         embedding=embeddings,
20         collection_name=VECTOR_COLLECTION_NAME
21     )

```

Listing 5: *rag_store.py*

5 Execution and Deployment Guide

5.1 Environment Setup

To initialize the project locally, execute the following commands in the terminal:

```
# Activate virtual environment
source venv/bin/activate

# Install requirements
pip install -r requirements.txt

# Run the Streamlit Application
streamlit run app.py
```

6 Acknowledgements

Special acknowledgment is extended to my project mentor, **[Insert Mentor Name]**, for continuous technical guidance throughout the architecture and deployment phases of this system.